

Experimental Learning of a Lift-Maximizing Central Pattern Generator for a Flapping Robotic Wing

Yagiz E. Bayiz¹, Shih-Jung Hsu¹, Aaron N. Aguiles¹, Yano Shade-Alexander¹ and Bo Cheng¹

Abstract— In this work, we present an application of a policy gradient algorithm to a real-time robotic learning problem, where the goal is to maximize the average lift generation of a dynamically scaled robotic wing at a constant Reynolds number (Re). Compared to our previous work, the merit of this work is two-fold. First, a central pattern generator (CPG) model was used as the motion controller, which provided a smooth generation and transition of rhythmic wing motion patterns while the CPG was being updated by the policy gradient, thereby accelerating the sample generation and reducing the total learning time. Second, the kinematics included three degrees of freedom (stroke, deviation, pitching) and were also free of half-stroke symmetry constraint, together they yielded a larger kinematic space which later explored by the policy gradient to maximize the lift generation. The learned wing kinematics used the full range of stroke and deviation to maximize the lift generation, implying that the wing trajectories with larger disk area and lower frequencies were preferred for high lift generation at constant Re . Furthermore, the wing pitching amplitude converged to values between 45° - 49° regardless of what the other parameters were. Notably, the learning agent was able to find two locally optimal wing motion patterns, which had distinct shapes of wing trajectory but generated similar cycle-averaged lift.

I. INTRODUCTION

The last two decades have witnessed rapid advances in both flapping flight literature and the development of a variety of hovering-capable flapping-wing Micro Air Vehicles (MAVs) [1], [2], [3] ranging in wing length from 1.5 mm [4] to 70 mm [5]. However, existing designs still fall behind to match the flight performance of flying insects such as stability, maneuverability and efficiency. Unlike high Reynolds Number (Re) flight, the biological fliers surpass the engineering designs in many aerodynamic performance measures at low Re regime ($10 - 10^4$).

In particular, lift generation capabilities of flapping flight have not been fully explored or optimized in the literature. Notably, several wing trajectory optimization studies have been performed in the past for low Re flapping flight using either quasi-steady [6], [7], [8], [9], [10] or low-order unsteady aerodynamic models [11] to estimate the forces and power used in the optimization. Despite the progress made, neither of these models are ideal for the optimization of wing trajectories. Although these models are computationally affordable, they approximate the force production of flapping wings to limited sets of wing kinematics as their validity

is local within certain region in the trajectory space. Furthermore, these models can potentially lead to exploitation of nonphysical mechanisms in the optimization, therefore potentially yielding false results [10]. As an alternative, computational fluid dynamics (CFD) solutions at low Re can accurately calculate the aerodynamic loads generated by a flapping wing. Recently, Lee and Lua [12] used CFD simulations in combination with quasi-steady models to optimize the pitching motion for a flapping wing. However, CFD simulations are computationally costly, thus often being impractical to be used in the optimization.

In contrast to low order modeling and CFD analysis, force measurements using dynamically scaled wings offer excellent accuracy with relatively fast experiments [13], [14], [15]. Although the large trajectory space available to flapping flight with multiple degrees of freedom often renders the full parametric studies using robotic wings infeasible, if paired with robot learning algorithms in a Hardware-in-the-loop (HIL) setting, these robotic setups have the potential to enable researchers to quickly identify at least the local optimal wing motion patterns in model-free settings. However, real-time learning in fluid environments is a relatively novel concept and is difficult to achieve due to the complexity fluid flow. Studies focusing on real-time flapping wing trajectory learning remain especially scarce. In one of the most relevant studies, Motamed and Jan [16] applied a simple Q-Learning algorithm [17] to maximize lift generation for a dynamically scaled robotic wing with only two degrees of freedom (stroke and pitching). In another study [18], a genetic algorithm is applied to optimize the kinematics of experimental flapping wing setup focusing solely on the pitching kinematics.

Our previous work [19] has demonstrated that policy gradient algorithms can identify near optimal hovering wing kinematics for efficiency maximization. Additionally, as an initial attempt to explore learning algorithms for the robotic wing setup, we kept the problem formulation rather simple, i.e., the motion was restrained to only two degrees of freedom (stroke and pitching) and the learning was performed on only two policy parameters. Furthermore, the generation of experimental samples in the learning process was relatively slow due to the inability of smooth transition between wing trajectories when the policy was updated.

In this work, we extend our previous efforts on applying policy gradient (PG) algorithm to the real-time robotic learning problem. Here we aim to maximize the average lift generation in the dynamically scaled flapping robotic wing at a constant $Re = 1200$, with three degrees of wing kinematic patterns being generated by a central pattern generator (CPG)

This research was supported by National Science Foundation (NSF CMMI 1554429), Army Research Office DURIP grant (W911NF1610272) and the Pennsylvania State University.

¹Department of Mechanical Engineering, Penn State University, University Park, PA 16802, USA. yeb3@psu.edu, buc10@psu.edu

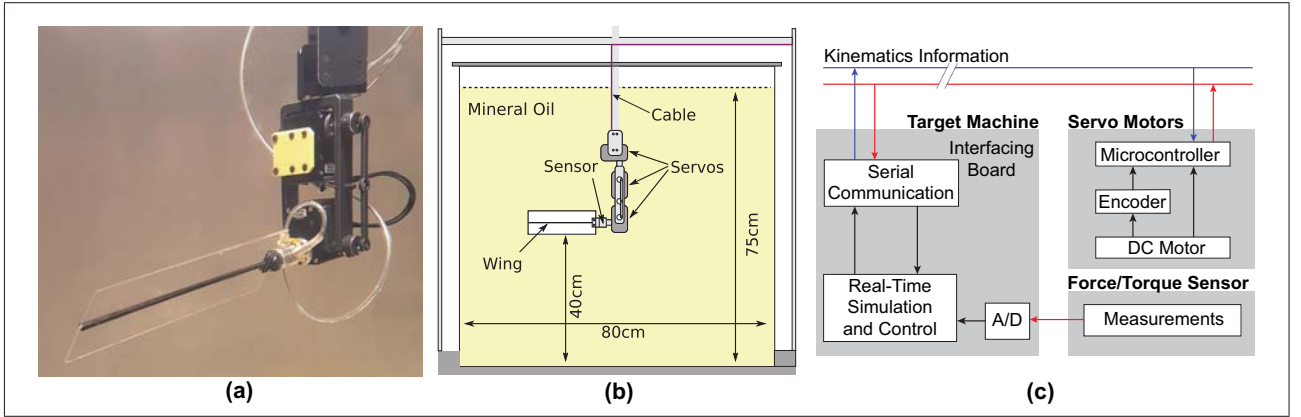


Fig. 1. (a) Dynamically scaled robotic wing. (b) Experimental setup. (c) The communication between the target machine, motors and sensor.

model. The parameters of the CPG model are the policy parameters, which are optimized via parameter exploring policy gradient algorithm (PEPG) [20] in a hardware-in-the-loop setting. Unlike our previous work [19], the wing kinematics for this application are rather complex, including six kinematic parameters and are also free of half-stroke symmetry constraint. In addition, the application of the CPG model as the motion controller, provides smooth generation and transition of rhythmic wing motion patterns while the CPG is being updated by the policy gradient, thereby accelerating the sample generation compared to the direct parameterization of wing kinematics used in [19].

The rest of this paper is organized as follows. In Section II, the configuration and control of robotic wing is explained. Section III is devoted to the construction of the CPG model and Section IV introduces the learning algorithm and describes the real-time learning problem. Finally, the learning results are presented and discussed in Section V.

II. DYNAMICALLY SCALED ROBOTIC WING

The dynamically scaled robotic wing (Fig. 1a) was actuated by three PID-embedded servo motors (XM450-W350-R, Robotis) with integrated DC motors, gear trains and magnetic encoders (resolution: $360^\circ/4096$). Each servo motor controlled one degree of freedom of wing motion: stroke (servo 1), deviation (servo 2) and pitching (servo 3). A six-axis force/torque load cell (Nano17-IP65, ATI) was mounted between the output shaft of servo 3, and the wing root measuring the force and moment acting on the entire wing. The wing was submerged into a $0.80\text{ m} \times 0.80\text{ m} \times 0.80\text{ m}$ mineral oil tank (White Mineral Oil, Tulco, density $\rho = 826\text{ kg m}^{-3}$, viscosity $\nu = 6 \times 10^{-6}\text{ m}^2\text{ s}^{-1}$ at 22°C) and fixed to the center of the tank (Fig. 1b).

A Speedgoat real-time target machine (Performance Real-Time Target Machine, Speedgoat GmbH) was used to control the robot. The control signals were generated by a Simulink[®] model and were transmitted to the servos using RS-485 protocol. The baud rate was set to 1MHz and the sampling frequency was at 250Hz. The servo tracking error of the tested reference trajectories was less than 1° . The

measurement range for force and torque were $\pm 12\text{ N}$ and $\pm 120\text{ N mm}$, with a resolution of $1/320\text{ N}$ and $1/64\text{ N mm}$, respectively. Additionally, the high frequency noise from the force and moment measurements were filtered with an online linear-phase FIR filter [21] enabling immediate evaluation of the wing trajectory performance.

The wing kinematics of the robotic wing (Fig. 2a) were described by three Euler angles, namely the stroke angle θ_1 (position of wing in horizontal stroke plane), the deviation angle θ_2 (wing deviation from the horizontal stroke plane), and the pitching angle θ_3 (angle of the wing about its radial axis). In this work, a rectangular wing (Fig. 2b) made of PETG material was used for the learning experiment. The wing had an aspect ratio of 3 and a pitching axis through the mid-chord. All parameters defining the wing geometry are provided in Table I.

III. CPG BASED TRAJECTORY GENERATION

In this work, the wing kinematics were generated using a CPG model. CPGs are networks of neural oscillators that produce rhythmic signals without receiving rhythmic inputs [22]. They have been applied to many robotics applications [23], [24], [25] requiring rhythmic locomotion control. CPGs possess excellent stability and limit cycle behavior and they require only a few control parameters [22]. The CPG was built upon amplitude-controlled phase oscillators derived from the Kuramoto oscillator model [26], [27], and the CPG network used here is similar to those applied to EPFL's robotic salamander [28], robotic snake [29] and robotic fish [30]. Each oscillator directly controlled one of the kinematic

TABLE I
PARAMETERS DEFINING THE WING GEOMETRY FOR EXPERIMENTS

Parameter	Description	Value
b	Wing platform span	180mm
Δr	Wing platform offset	55mm
c	Chord length	60mm
h_{wing}	Wing thickness	1.5mm
x_{rot}	Distance of pitching axis from leading edge	30mm

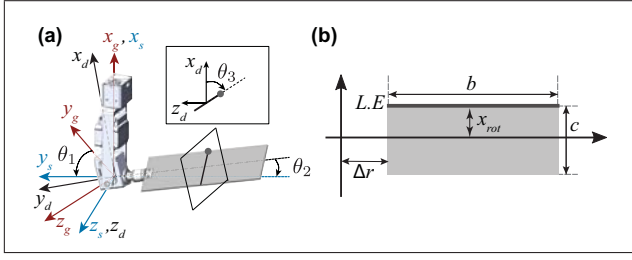


Fig. 2. (a) Coordinate frame. Subscripts g (global), s (stroke), d (deviation) denote the corresponding frames. Gravity is towards $-x_g$ direction. (b) Schematic of the wing showing the parameters. L.E shows the leading edge.

variables, namely θ_1, θ_2 and θ_3 , and the dynamics of the oscillator i can be described as,

$$\dot{\phi}_i = 2\pi f_i + \sum_j w_{ij} m_j \sin(\phi_j - \phi_i - \psi_{ij}), \quad (1a)$$

$$\ddot{m}_i = \frac{\alpha_m^2}{4} (M_i - m_i) - \alpha_m \dot{m}_i, \quad (1b)$$

$$\ddot{x}_i = \frac{\alpha_x^2}{4} (X_i - x_i) - \alpha_x \dot{x}_i, \quad (1c)$$

$$\theta_i = x_i + m_i \sin \phi_i, \quad (1d)$$

where the internal state variables ϕ_i, m_i and x_i are the instantaneous phase, amplitude and the offset of the oscillator output θ_i respectively; and f_i, M_i and X_i denote the desired values for the frequency, amplitude and offset of the rhythmic outputs, respectively. The two coupling parameters w_{ij} and the ψ_{ij} express the effects of other oscillators on oscillator i where w_{ij} is the coupling weight and ψ_{ij} is the phase bias between oscillators i and j . Finally, positive constants α_m and α_x are the decay rates for the critically damped amplitude dynamics and determine the rate of convergence for m_i and x_i to M_i and X_i respectively.

The oscillator structure used in this application is presented in Fig. 3a. The desired frequency of oscillation for each kinematic variable was set to be identical, i.e. $f_i = f \forall i$. Also, for this particular study, $\alpha_x = \alpha_m = 5\text{Hz}$ was selected to achieve fast convergence. The deviation θ_2 and the pitching θ_3 had no direct connection with each other and these oscillators were separately synchronized to the stroke servo. As a result, the coupling weights were chosen as $w_{23} = w_{32} = 0, w_{12} = w_{13} = w_{21} = w_{31} = 0.1$. There was no self-coupling in the oscillators, meaning that $w_{ii} = 0$. Finally, the phase bias terms between the stroke and the other two angles were determined as $\psi_{12} = \Delta\phi_2, \psi_{21} = -\Delta\phi_2, \psi_{13} = \Delta\phi_3, \psi_{31} = -\Delta\phi_3$, while $\psi_{ij} = 0$ for all other cases. The $\Delta\phi_i$ terms represent the desired phase difference between the stroke and the other kinematic angles.

Given the settings above, the oscillators asymptotically converge to the following limit cycle θ_i^∞ [28], [29].

$$\theta_i^\infty(t) = X_i + M_i \sin(2\pi f t + \Delta\phi_i + \phi^0), \quad (2)$$

where t is continuous time and ϕ^0 is shared phase for all oscillators, determined by the initial conditions of the CPG network. This limit cycle behavior was assumed to be the

parametrization of our steady kinematic variables θ_1, θ_2 and θ_3 , i.e. policy parametrization and the policy parameters (see Section IV-B) were selected among parameters defining the characteristics of the harmonic motion, f, M_i, X_i and $\Delta\phi_2$ (see Section IV-A for details).

One particular advantage of this limit cycle behavior for this learning application was that it allowed smooth and fast transition between different wing kinematic patterns when there were instant changes in reference parameters as a result of policy updates. In this application, with the selected parameters, the transition from one limit cycle to another was observed to be faster than half a stroke cycle. Unlike our previous work [19], with the CPG based wing kinematics generator, the robotic wing did not have to stop every time the parameter set was updated. Additionally, we observed that the fluid forces applied on the robotic wing settled to a steady-state within two strokes cycles, faster than the previous application, potentially due to already existed wake dynamics in the flow. As a result, the clock time of the learning experiment was reduced solely due to the increased speed in generating samples needed for learning, as compared with those in [19].

IV. LEARNING PROBLEM AND ALGORITHM

A. Learning Problem

In this work, the learning problem was formulated to investigate the lift generation potential of the hovering flapping wing at a constant $Re = 1200$. The optimized parameter vector $\mathbf{v}$ for this application is selected as $\mathbf{v} = [M_1 \ M_2 \ M_3 \ X_3 \ \Delta\phi_2 \ \Delta\phi_3]^T$ where the wing kinematics are generated using (1). This set of parameters were capable of generating wide range of wing kinematics including out of plane motion for stroke and kinematics with asymmetrical downstroke and upstroke. Note that, the offsets for stroke and deviation, X_1 and X_2 were excluded from the parameter set to 0. The frequency, f was also not a part of the learning parameters since f was adjusted for each parameter set to maintain $Re = 1200$ using the definition of Re .

$$Re = \frac{c \bar{U}_{r_2}}{\nu} = \frac{c r_2 f \int_0^{1/f} \sqrt{\dot{\theta}_1^2 + \dot{\theta}_2^2} dt}{\nu}, \quad (3)$$

where $\bar{U}_{r_2}$ is the time averaged velocity at $\hat{r}_2$ (second moment of wing area, located at 159mm away from the wing root for this case). Since all the other fluid and geometrical parameters were the same, the cycle-averaged velocity was the key design parameter that was kept constant.

The goal in this study was to maximize the cycle-averaged lift which can be expressed as:

$$\mathcal{R}(\mathbf{v}) = \bar{F}_{L,meas}(\mathbf{v}) = \int_0^{1/f} F_{L,meas}^y(t) dt, \quad (4)$$

where $F_{L,meas}^y(t)$ is the instantaneous lift following $\mathbf{v}$ and the $\mathcal{R}(\mathbf{v})$ is the performance of the wing following trajectory defined by $\mathbf{v}$ (return in learning the framework, see Section IV-B). The constraints on the learning parameters are tabulated in Table II. These boundaries were caused

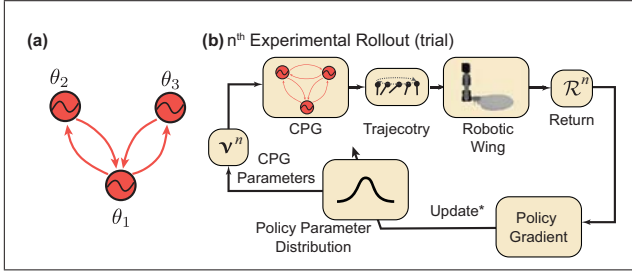


Fig. 3. (a) CPG structure; circles represent oscillators for the kinematic variables and arrows denotes the coupling between oscillators. (b) Learning framework as a block diagram. Update was performed after all rollouts were actuated and returns $\mathcal{R}(\mathbf{v}^n)$, collected.

by mostly mechanical constraints with the exception of the upper limit of M_1 , which was based on insect flight literature. To obtain a fully developed, steady flow field, the measurements started subsequent to the end of the second stroke cycle. Note that, after the transition to the updated wing kinematic pattern, the new wake developed faster than that of a wing starting from the stationary, due to the existing wake from the previous wing kinematic pattern, and therefore we only waited two wing beats before each force measurement. Furthermore, the force averaging was performed for two full cycles.

B. Policy Gradient Methods

Reinforcement learning (RL) framework consists of a distinct environment and an agent interacting with the environment at every time step k . Based on the state of the agent $\mathbf{x}_k$ and the action $\mathbf{u}_k$ drawn from a policy π , this interaction results in a scalar value measuring the instantaneous performance of the agent called reward r_k . Jointly, all the states experienced and actions taken by the robot form a trajectory $\tau = (\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{x}_K, \mathbf{u}_K)$, which is also known as the rollout, where K is the terminal time step [31]. In RL framework the goal of the agent is to seek for an optimal π that maximizes a function of cumulative aggregation of rewards collected over the trajectories. This accumulative function is the return, $\mathcal{R}$, and unlike rewards these returns are a measure of long-time performance of the policy. Since the setup is usually stochastic, the agent tries to maximize the expected value of $\mathcal{R}$ following policy π that is J^π .

To maximize J^π , PG methods use parameterized policies $\pi_{\mathbf{v}}$ where $\mathbf{v}$ denotes the parameter vector for the policy. Thus, PG algorithms directly operate in the parameter space of parameterized policies. The update of policy is directly based on sampled trajectories τ^n and $\mathcal{R}(\tau^n)$ where n denotes

the index of the trajectory. These methods try to update the parameters using gradient ascent for maximizing the expected return J^π . Therefore, an estimation of the policy gradient, $\nabla_{\mathbf{v}} J^\pi$ is required to determine the direction of steepest ascent of the expected return.

In classical likelihood-ratio policy gradient algorithms [32], a numerical estimation for the policy gradient is found $\nabla_{\mathbf{v}} J^\pi$. For these algorithms, the deterministic $\mathbf{v}$ that directly determines the policy $\pi_{\mathbf{v}}$ is learned and the exploration is directly applied to the action $\mathbf{u}_k$ drawn from $\pi_{\mathbf{v}}$. However, sampling from the policy on every time step leads to a high variance in the sample over histories. This leads to a noisy gradient estimate [20]. Another alternative is to use an algorithm that learns a probability distribution for the parameter vector $\mathbf{v}$ instead of the parameter vector itself. Thus, the exploration is incorporated to parameter space instead of the action space. These algorithms often considered to belong in Evolutionary Strategies (ES) [33], since a $\mathbf{v}^n$ is drawn at the start of each rollout (thus a member of a generation is picked) and the final $\mathcal{R}^n$ (fitness) for that $\mathbf{v}^n$ is used to determine the policy gradient after all the rollouts are completed. An example for such methods is the Natural Evolutionary Strategies (NES) [34], [35], [36].

In this study, we used a simple variant of NES methods called PEPG algorithm [37], [20]. The major benefit of PEPG algorithm was its robustness and easy implementation to train the wing trajectories which are essential in real-time robotic experiments. Additionally, since the parameters were updated after the whole trajectory is completed, partial observability of fluid problems resulted from the history dependence of the flow field did not affect the learning speed. Unlike fluid applications with value-based methods [38], this algorithm did not require any recurrent networks or other treatments to converge. One drawback of using PEPG in experimental work was that the collected data samples were used extravagantly and each sample was used only for one update since it is an on-policy [39] method. For experimental application of this kind, where the sample generation is slow and costly, the sample efficiency plays a crucial role in convergence rates. However, the simplicity and robustness of PEPG outweighed this issue in the current problem as the experiment clock times were still in acceptable range. Additionally, an extension of PEPG [40] has been proposed that extends the algorithm with importance sampling [41], providing more efficient use of the collected samples. In this particular study, our experiments suggested that the PEPG with or without importance sampling have similar convergence rates. Thus, we used the regular PEPG in the current application.

C. Parameter Exploring Policy Gradient Algorithm

In PEPG, at the beginning of each trajectory, the policy parameters $\mathbf{v}$ are sampled from the prior distribution $p(\mathbf{v}|\boldsymbol{\kappa})$ where $\boldsymbol{\kappa}$ is the hyperparameter vector for that distribution, and thereafter the policy is deterministic. Thus, the variance in gradient estimate is reduced since the parameters are sampled once per trajectory. To maximize the expected

TABLE II
CONSTRAINTS FOR LEARNING PARAMETERS

Variable	M_1	M_2	M_3	X_3	$\Delta\phi_2$	$\Delta\phi_3$
Upper Limit	90°	20°	90°	45°	∞	∞
Lower Limit	30°	0°	0°	-30°	$-\infty$	$-\infty$

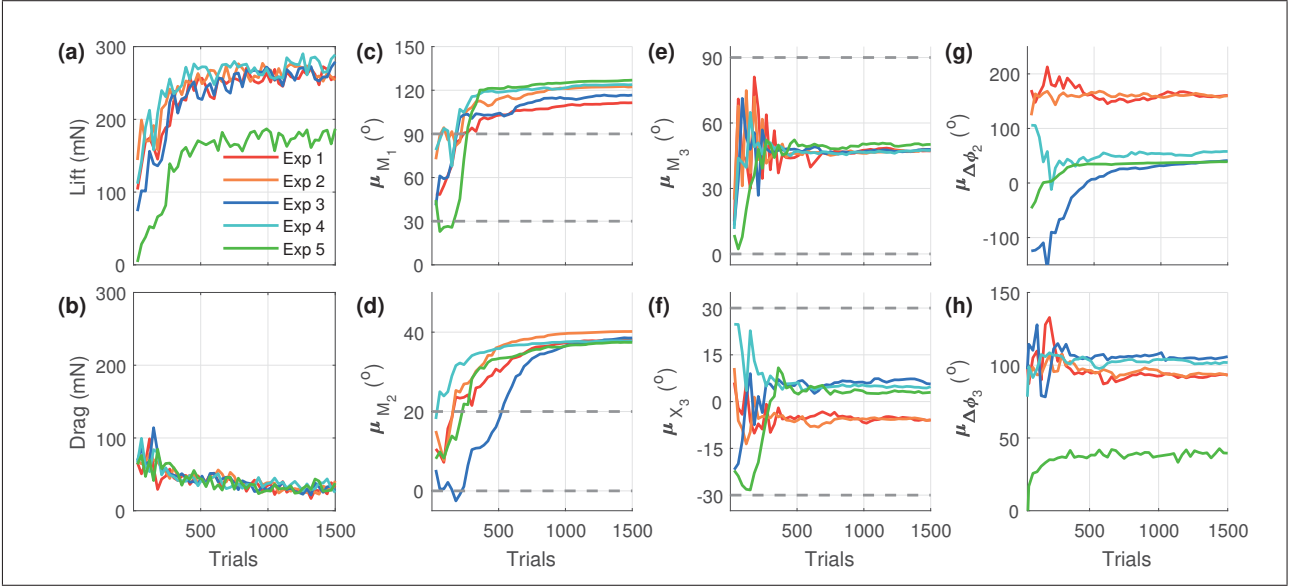


Fig. 4. Learning performance and change in means of parameters over time. Individual experiments are labeled as Exp 1-5 where each experiment is represented by a color. (a) Progression of average generated lift at each episode. (b) The root mean square of the corresponding cycle-averaged drag. (c)-(h) Convergence of the means of the policy parameter distributions. Dash lines mark the upper and lower boundaries for the drawn policy parameter. The phase figures (g)-(h) are shifted by integer multiples of 360° for consistency.

return, the hyperparameters can be updated as;

$$\kappa \leftarrow \kappa + \gamma \nabla_{\kappa} J^{\kappa} \quad (5)$$

where γ is the learning rate. For the sake of brevity, the derivation of the estimation for J^{κ} will not be presented here, the details of which can be found in [20]. Using the maximum likelihood tricks, $\nabla_{\kappa} J^{\kappa}$ can be derived to the following final form.

$$\nabla_{\kappa} J^{\kappa} \approx \frac{1}{N} \sum_{n=1}^N \nabla_{\kappa} \log p(\mathbf{v}^n | \kappa) \mathcal{R}(\mathbf{v}^n) \quad (6)$$

where N denotes the number of rollouts (samples) and $\mathcal{R}(\mathbf{v}^n)$ is the collected return following the trajectory resulted from parameter $\mathbf{v}^n$.

For this particular work, unlike the commonly used normal distributions, we used univariate truncated normal distributions [42] to draw the parameters in $\mathbf{v}$. The reason for this selection was to naturally implement parameter boundaries resulting from constraints defined in learning problem (see Section IV-A). Since these boundaries were fixed for all parameters, the hyperparameter vector can be defined as, $\kappa = [\mu \ \sigma]^T$, where μ and σ represents mean and standard deviation vectors of the truncated normal distributions.

As most likelihood-ratio policy gradient algorithms, PEPG benefits from a constant baseline subtracted from the rewards[43]. The purpose of this baseline is to reduce the variance without inducing a bias to the gradient estimate, thus speeding up the convergence to true policy gradient with low number of rollouts. Although the baseline can be selected arbitrarily, it is usually designed to minimize the variance on gradient estimator. Such an optimal baseline for PEPG was derived in [44], which was also adopted in this study.

V. EXPERIMENTS AND RESULTS

In this section, the results of the learning experiments are presented. The learning was repeated 5 times with each experiment initialized with random μ_0 and fixed standard deviations $\sigma_0 = [15^\circ \ 10^\circ \ 15^\circ \ 15^\circ \ 20^\circ \ 20^\circ]^T$. The learning rates were manually tuned ($\gamma_{\mu} = 0.7$ and $\gamma_{\sigma} = 0.07$) to update μ and σ respectively. Note that, these learning rates were not optimal and can be further tuned for better convergence rates. The number of rollouts per episode was determined empirically as 30 (i.e. hyperparameter vector κ was updated after 30 rollouts) resulting in sufficiently fast convergence rates for all the cases.

The learning results showing the convergence over 50 episodes (or 1500 rollouts) for each experiment is provided in Fig. 4. The experiment time span varied from 15 to 16 hours for the completion of the 50 episodes. Note that, the required time for the same experiment with the direct parameterization method used in [19] would be 22 to 23 hours. Fig. 4a presents the average of $\mathcal{R}$ (cycle-averaged lift) per each episode which converges within 20 episodes for all experiments as the average lift plateaus afterwards. As seen from the figure, learning curves for the experiments (except Exp 5) all converged to a similar final value (around 270mN), indicating a near local optimal result was reached.

The progression of the mean values for policy parameter distributions are provided in Fig. 4c-h. The mean values for stroke amplitude M_1 and deviation amplitude M_2 quickly exceeded their upper limits (Table II). Therefore, these parameters were only selected from the tail of the respective truncated normal distributions, meaning that M_1 and M_2 are extremely likely to be drawn close to their upper limits. Clearly, the agent tried to maximize the amplitude of both

stroke and deviation, i.e. the arc traveled by the wing was maximized. This might be an effort to increase the effective disk area, a strong indicator of the aerodynamic efficiency [45]. The mean value of the pitching amplitude M_3 distribution (Fig. 4e) also consistently converged to between 45° to 49° for all the experiments regardless the convergent values of other hyperparameters. Notably, this result reveals a favorable pitching amplitude shared by different flapping strategies which could be used as a rule of thumb in design of flapping wing MAVs.

Examining the distribution means for the rest of the policy parameters uncovers two locally optimal high lift-generation strategies with categorically different wing kinematic patterns. As shown in Fig. 4f-h, among the experiments with high lift generation, Exp 1 and 2 (red group) converged to similar final values for μ_{X_3} , $\mu_{\Delta\phi_2}$ and $\mu_{\Delta\phi_3}$, whereas there was a clear resemblance between Exp 3 and 4 (blue group) for the same parameters. Specifically, for the pitching offset X_3 the Exp 1 and 2 converged to a small negative angle (-6°) while Exp 3 and 4 converged to 4° positive offset. In addition, the phase between stroke and deviation $\Delta\phi_2$ (Fig. 4g), converged to approximately at 160° for Exp 1 and 2 whereas it converged to $50^\circ < \mu_{\Delta\phi_2} < 60^\circ$ for Exp 3 and 4. Finally, the converged values for distribution mean of the phase parameter between stroke and pitching $\Delta\phi_3$, converged to approximately 90° in Exp 1 and 2 while for Exp 3 and 4, this value settled to approximately 105° (Fig. 4h). Despite the significant differences between these two wing kinematic patterns, they resulted in similar values of cycle-averaged lift (Fig. 4a), suggesting that they were distinct local optima with similar performances. Note that, the final policy parameter distributions of Exp 5 were almost identical to those in Exp 3 and 4 except $\Delta\phi_3$. The wing pitching for Exp 5 converged to a poor local optimum with highly delayed wing rotation at stroke reversal (see Fig. 5e,h), thus leading to an significantly lower learning performance.

The best performing wing kinematic trajectories from the final episode for all the experiments are shown in Fig. 5. Although being considerably different, trajectories from both red (Exp 1 and 2) and blue groups (Exp 3 and 4) resemble to those commonly observed in natural fliers capable of sustained hovering [46]. However, the differences between these two trajectory groups may have led to distinct aerodynamic mechanisms for lift generation. The most prominent difference is the deviation characteristics of the trajectories. For Exp 1 and 2, the stroke plane was almost flat while being tilted. In contrast, the deviation introduces ovality to the trajectories in Exp 3 and 4, which resulted in relatively horizontal stroke plane with the majority of the deviation change occurred at stroke reversals. Another disparity between these groups was simply the range of wing pitching (Fig. 5h). Exp 1 and 2, demonstrated a slightly larger range for wing pitching and there was a visible pitching offset difference between the two groups.

Another interesting result for this experiment comes from Fig. 4b, which presents the root mean square (RMS) of all the cycle-averaged drag measurements for rollouts. The

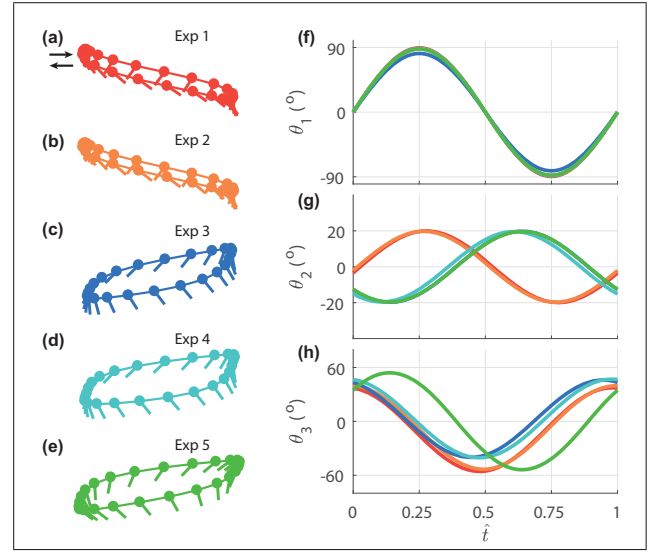


Fig. 5. Limit-cycle kinematics of the best parameter set at the final episode of each experiment. (a)-(e) Projections of the 3D kinematics of a chord slice on 2D plane. (f)-(h) Instantaneous values of stroke θ_1 (f), deviation θ_2 (g), and pitching θ_3 (h).

RMS evaluation signifies the magnitude of the generated drag at each episode and this value diminishes for all the experiments as learning progresses. Consequently, for Exp 1-4 (high lift generating experiments) the drag generation is observed to be significantly lower compared to the lift (less than 10%). Therefore, cycle-averaged drag is maintained to be relatively small even though the final wing trajectories are clearly asymmetrical between half strokes.

VI. CONCLUSIONS AND FUTURE WORK

This work demonstrated an application of a policy gradient (PEPG) algorithm to an online lift maximization problem for a dynamically scaled robotic wing at $Re = 1200$ hovering flight. In comparison to our previous work [19], this work included all three degrees of freedom of the wing kinematics in the learning problem. Additionally, the wing kinematic patterns were generated by a CPG model, which reduced the time span of the learning experiments by improving the sample generation speed. Our results imply that deviation can play an important role in lift generation which is often overlooked in literature of wing kinematics optimization. Furthermore, the results presented here may escalate the wing kinematics design process for MAVs as the key wing kinematics features for lift maximization were provided.

As a preliminary effort to improve our previous work, we kept the number of experiments small in this study which limits the validity of the results. Thus, in the future, we will develop this work with more repetitions. Furthermore, the effects of the deviation will be further investigated by expanding the wing kinematics space, either by using better parametrizations or relaxing the mechanical constraints. Finally, we will pursue methods to improve the learning convergence rate by further improving the sample generation speed or the sample efficiency in the learning algorithm.

REFERENCES

- [1] R. J. Wood, "The first takeoff of a biologically inspired at-scale robotic insect," *IEEE transactions on robotics*, vol. 24, no. 2, pp. 341–347, 2008.
- [2] M. Keennon, K. Klingebiel, and H. Won, "Development of the nano hummingbird: A tailless flapping wing micro air vehicle," in *50th AIAA Aerospace Sciences Meeting including the New Horizons Forum and Aerospace Exposition*, 2012, p. 588.
- [3] J. Zhang, B. Cheng, and X. Deng, "Instantaneous wing kinematics tracking and force control of a high-frequency flapping wing insect mav," *Journal of Micro-Bio Robotics*, vol. 11, no. 1-4, pp. 67–84, 2016.
- [4] J. Bronson, J. Pulskamp, R. Polcawich, C. Kroninger, and E. Wetzel, "Pzt mems actuated flapping wings for insect-inspired robotics," in *Micro Electro Mechanical Systems, 2009. MEMS 2009. IEEE 22nd International Conference on*. IEEE, 2009, pp. 1047–1050.
- [5] L. Hines, D. Campolo, and M. Sitti, "Lift-off of a motor-driven, flapping-wing microaerial vehicle capable of resonance," *IEEE Transactions on Robotics*, vol. 30, no. 1, pp. 220–232, 2014.
- [6] G. J. Berman and Z. J. Wang, "Energy-minimizing kinematics in hovering insect flight," *Journal of Fluid Mechanics*, vol. 582, pp. 153–168, 2007.
- [7] M. Kurdi, B. Stanford, and P. Beran, "Kinematic optimization of insect flight for minimum mechanical power," *AIAA paper*, vol. 1420, 2010.
- [8] H. E. Taha, M. R. Hajj, and A. H. Nayfeh, "Wing kinematics optimization for hovering micro air vehicles using calculus of variation," *Journal of Aircraft*, 2013.
- [9] X. Ke, W. Zhang, X. Cai, and W. Chen, "Wing geometry and kinematic parameters optimization of flapping wing hovering flight for minimum energy," *Aerospace Science and Technology*, vol. 64, pp. 192–203, 2017.
- [10] Y. Bayiz, M. Ghanaatpishe, H. Fathy, and B. Cheng, "Hovering efficiency comparison of rotary and flapping flight for rigid rectangular wings via dimensionless multi-objective optimization," *Bioinspiration & biomimetics*, vol. 13, no. 4, p. 046002, 2018.
- [11] Z. Yan, H. E. Taha, and M. R. Hajj, "Effects of aerodynamic modeling on the optimal wing kinematics for hovering mavs," *Aerospace Science and Technology*, vol. 45, pp. 39–49, 2015.
- [12] Y. Lee and K. Lua, "Optimization of simple and complex pitching motions for flapping wings in hover," *AIAA Journal*, vol. 56, no. 6, pp. 2466–2470, 2018.
- [13] M. H. Dickinson, F.-O. Lehmann, and S. P. Sane, "Wing rotation and the aerodynamic basis of insect flight," *Science*, vol. 284, no. 5422, pp. 1954–1960, 1999.
- [14] B. Cheng, J. Roll, Y. Liu, D. R. Troolin, and X. Deng, "Three-dimensional vortex wake structure of flapping wings in hovering flight," *Journal of The Royal Society Interface*, vol. 11, no. 91, p. 20130984, 2014.
- [15] L. Chen, J. Wu, C. Zhou, S.-J. Hsu, and B. Cheng, "Unsteady aerodynamics of a pitching-flapping-perturbed revolving wing at low reynolds number," *Physics of Fluids*, vol. 30, no. 5, p. 051903, 2018.
- [16] M. Motamed and J. Yan, "A reinforcement learning approach to lift generation in flapping mavs: Experimental results," in *Robotics and Automation, 2007 IEEE International Conference on*. IEEE, 2007, pp. 748–754.
- [17] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3, pp. 272–292, 1992.
- [18] A. Gehrke, G. de Guyon-Crozier, and K. Mulleners, "Genetic algorithm based optimization of wing rotation in hover," *Fluids*, vol. 3, no. 3, p. 59, 2018.
- [19] Y. E. Bayiz, L. Chen, S.-J. Hsu, P. Liu, A. N. Aguilés, and B. Cheng, "Real-time learning of efficient lift generation on a dynamically scaled flapping wing using policy search," in *Robotics and Automation (ICRA), 2018 IEEE International Conference on*. IEEE, 2018.
- [20] F. Sehnke, C. Osendorfer, T. Rückstieß, A. Graves, J. Peters, and J. Schmidhuber, "Parameter-exploring policy gradients," *Neural Networks*, vol. 23, no. 4, pp. 551–559, 2010.
- [21] N. J. Fliege, *Multirate digital signal processing*. John Wiley New York, 1994, vol. 994.
- [22] A. J. Ijspeert, "Central pattern generators for locomotion control in animals and robots: a review," *Neural networks*, vol. 21, no. 4, pp. 642–653, 2008.
- [23] A. Spröwitz, A. Tuleu, M. Vespignani, M. Ajallooeian, E. Badri, and A. J. Ijspeert, "Towards dynamic trot gait locomotion: Design, control, and experiments with cheetah-cub, a compliant quadruped robot," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 932–950, 2013.
- [24] C. Liu, D. Wang, and Q. Chen, "Central pattern generator inspired control for adaptive walking of biped robots," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 43, no. 5, pp. 1206–1215, 2013.
- [25] X. Niu, J. Xu, Q. Ren, and Q. Wang, "Locomotion learning for an anguilliform robotic fish using central pattern generator approach," *IEEE Transactions on Industrial Electronics*, vol. 61, no. 9, pp. 4780–4787, 2014.
- [26] Y. Kuramoto, "Self-entrainment of a population of coupled non-linear oscillators," in *International Symposium on Mathematical Problems in Theoretical Physics*, H. Araki, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1975, pp. 420–422.
- [27] —, *Chemical oscillations, waves, and turbulence*. Springer Science & Business Media, 1984.
- [28] A. J. Ijspeert, A. Crespi, D. Ryczko, and J.-M. Cabelguen, "From swimming to walking with a salamander robot driven by a spinal cord model," *science*, vol. 315, no. 5817, pp. 1416–1420, 2007.
- [29] A. Crespi and A. J. Ijspeert, "Online optimization of swimming and crawling in an amphibious snake robot," *IEEE Transactions on Robotics*, vol. 24, no. 1, pp. 75–87, 2008.
- [30] A. Crespi, D. Lachat, A. Pasquier, and A. J. Ijspeert, "Controlling swimming and crawling in a fish robot using a central pattern generator," *Autonomous Robots*, vol. 25, no. 1-2, pp. 3–13, 2008.
- [31] M. P. Deisenroth, G. Neumann, and J. Peters, "A survey on policy search for robotics," *Foundations and Trends® in Robotics*, vol. 2, no. 1–2, pp. 1–142, 2013.
- [32] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," in *Machine Learning*, 1992, pp. 229–256.
- [33] H.-G. Beyer and H.-P. Schwefel, "Evolution strategies—a comprehensive introduction," *Natural computing*, vol. 1, no. 1, pp. 3–52, 2002.
- [34] D. Wierstra, T. Schaul, J. Peters, and J. Schmidhuber, "Natural evolution strategies," in *Evolutionary Computation, 2008. CEC 2008. (IEEE World Congress on Computational Intelligence)*. IEEE Congress on. IEEE, 2008, pp. 3381–3387.
- [35] Y. Sun, D. Wierstra, T. Schaul, and J. Schmidhuber, "Stochastic search using the natural gradient," in *ICML*, 2009, pp. 1161–1168.
- [36] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv preprint arXiv:1703.03864*, 2017.
- [37] F. Sehnke, C. Osendorfer, T. Rückstieß, A. Graves, J. Peters, and J. Schmidhuber, "Policy gradients with parameter-based exploration for control," *Artificial Neural Networks-ICANN 2008*, pp. 387–396, 2008.
- [38] S. Verma, G. Novati, and P. Koumoutsakos, "Efficient collective swimming by harnessing vortices through deep reinforcement learning," *Proceedings of the National Academy of Sciences*, p. 201800923, 2018.
- [39] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [40] T. Zhao, H. Hachiya, V. Tangkaratt, J. Morimoto, and M. Sugiyama, "Efficient sample reuse in policy gradients with parameter-based exploration," *Neural computation*, vol. 25, no. 6, pp. 1512–1547, 2013.
- [41] R. Y. Rubinstein and D. P. Kroese, *Simulation and the Monte Carlo method*. John Wiley & Sons, 2016, vol. 10.
- [42] N. Johnson, S. Kotz, and N. Balakrishnan, *Continuous Univariate Distributions, vol. 1*. John Wiley & Sons, 1994.
- [43] J. Peters and S. Schaal, "Policy gradient methods for robotics," in *Intelligent Robots and Systems, 2006 IEEE/RSJ International Conference on*. IEEE, 2006, pp. 2219–2225.
- [44] T. Zhao, H. Hachiya, G. Niu, and M. Sugiyama, "Analysis and improvement of policy gradient estimation," in *Advances in Neural Information Processing Systems*, 2011, pp. 262–270.
- [45] M. R. Nabawy and W. J. Crowther, "On the quasi-steady aerodynamics of normal hovering flight part i: the induced power factor," *Journal of The Royal Society Interface*, vol. 11, no. 93, p. 20131196, 2014.
- [46] S. N. Fry, R. Sayaman, and M. H. Dickinson, "The aerodynamics of hovering flight in drosophila," *Journal of Experimental Biology*, vol. 208, no. 12, pp. 2303–2318, 2005.